Lesson 14: Raising the Stakes

GameMaker

VIDEO GAME CREATION

This week: later waves get genuinely harder - more enemies, moving faster - using the exact wave variable you've had since Lesson 11.

Open your own [YourName]Game project from Lesson 13 - don't start a new one.

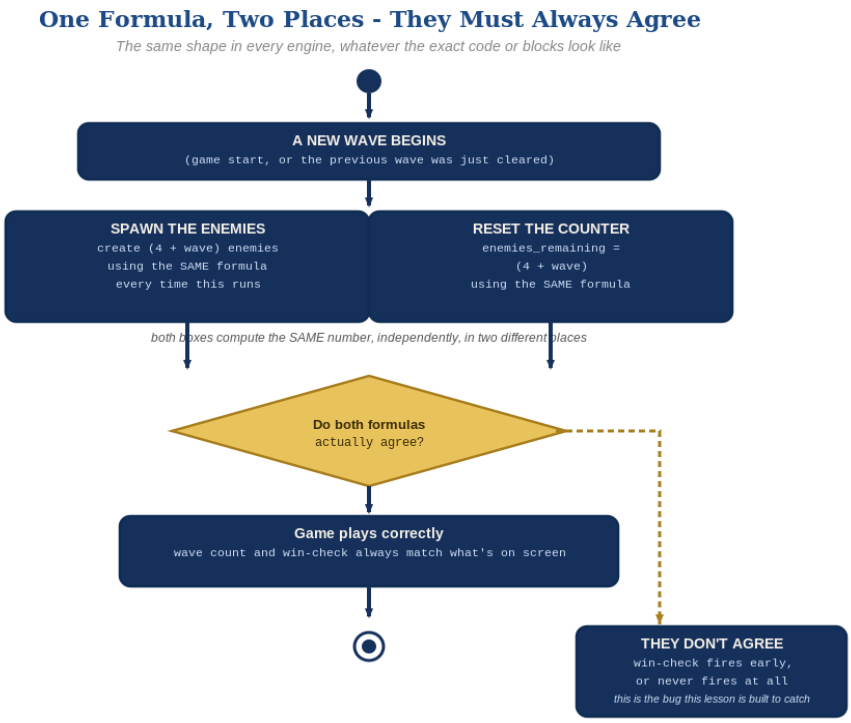
Tip: no new assets this week - difficulty scaling reuses your exact existing enemy sprites and spawn logic.

Setup

1. Open your [YourName]Game project from Lesson 13.
2. Add a new instance variable to obj_enemy's Create event: enemy_speed = 1;.

Level 1 - Minimal (MVP)

Each wave spawns one more enemy than the last (5, 6, 7). Speed stays the same for now - that's Level 2.



Whether it's two places (most engines) or the loop and the reset living in genuinely different scripts (Godot, GameMaker), the rule is the same: one formula, referenced everywhere.

VIDEO GAME CREATION

Whether the spawn loop and the reset live in two places or two genuinely different scripts, the rule is the same: one formula, referenced everywhere.

3. In spawn_enemies() (its own Script resource, scr_spawn_enemies), change the fixed "repeat (5)" loop into "repeat (4 + obj_gamemanager.wave)" (code below) - wave 1 spawns 5, wave 2 spawns 6, wave 3 spawns 7. spawn_enemies() is a free-standing global function, not owned by obj_gamemanager, so wave needs the obj_gamemanager. prefix here.
4. Wherever enemies_remaining gets reset to 5 for a new wave - inside start_game() and inside obj_bullet's win-check else branch - change that to 4 + wave too (that line already runs after wave is set/advanced in both spots, so no reordering needed) - all three places (the spawn loop and both resets) must always agree.
5. Press F5 (Run) and play through a full round - wave 2 should visibly spawn one more enemy than wave 1, and wave 3 one more than that.

Highlighted lines are new this week; everything else is code you already have.

```
// spawn_enemies() in scr_spawn_enemies (revised - count only, unchanged
contract):
function spawn_enemies() {
    repeat (4 + obj_gamemanager.wave) {
        var inst = instance_create_layer(x, y, "Instances", obj_enemy);
        // ...existing per-enemy positioning...
    }
}

// scr_start_game's start_game() (revised, enemies_remaining line only):
```

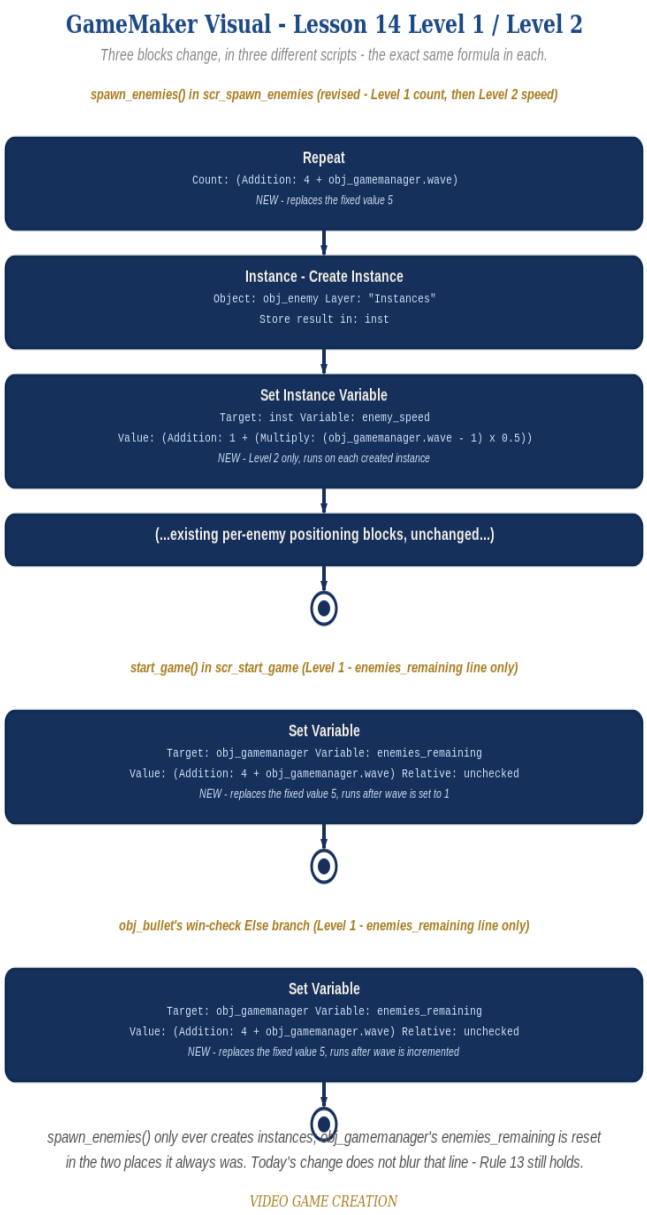
```
obj_gamemanager.enemies_remaining = 4 + obj_gamemanager.wave;

// obj_bullet's win-check else branch (revised, enemies_remaining line
only):
obj_gamemanager.enemies_remaining = 4 + obj_gamemanager.wave;
```

Alternate Path: GML Visual (no typing)

The same wave-dependent formulas work as drag-and-drop math blocks instead of typed code.

6. In scr_spawn_enemies, on spawn_enemies()'s Repeat block, replace the fixed value 5 with an Addition block combining 4 and a "Variable" block (Target obj_gamemanager, Variable wave).
7. On each created instance, add a "Set Instance Variable" block for enemy_speed, fed by an Addition block combining 1 and (a Variable block for obj_gamemanager.wave, minus 1) times 0.5.



spawn_enemies(), start_game(), and obj_bullet's win-check Else branch - three blocks change, in three different scripts, the exact same formula in each.

Level 2 - Complete (Alpha/Beta)

Enemy speed ALSO increases each wave, on top of the growing count. Restarting correctly returns everything to wave 1's original values.

- In spawn_enemies(), right after creating each enemy instance, set its speed based on the wave: inst.enemy_speed = 1 + (obj_gamemanager.wave - 1) * 0.5;.
- In obj_enemy's Step event, change "y += 1;" to "y += enemy_speed;" - the enemy now reads its own speed instead of a fixed literal.
- Restart mid-game and confirm both count and speed return correctly to wave 1's values.

- Play a full round and confirm wave 3 is noticeably busier and faster than wave 1, but still fair.

Level 3 - Refined - Choose ONE Stretch Feature

Pick exactly one of the five ideas below. These are intentionally open-ended - a good-faith attempt at the general idea is a complete Level 3 finish, not a fully polished feature.

- Destructible shields/barriers: a few simple blocks placed between the player and the enemies that disappear (or shrink) after absorbing a hit or two - reuses your existing collision-detection and hit-reaction patterns from Lessons 4 and 13, just on a new kind of object.
- A power-up: an item that occasionally drops or spawns, which the player collects to temporarily change how shooting works (faster fire rate, multiple bullets at once, or a brief shield) - reuses your existing timer/cooldown pattern from Lesson 3.
- A bonus enemy: a rare, faster-moving, or oddly-patterned enemy worth extra points, spawned occasionally alongside a normal wave - reuses this week's own speed/count scaling ideas, just for one special case instead of the whole wave.
- Formation-marching movement: instead of each enemy independently drifting down (this course's approach since Lesson 4), have the whole wave move together as a group and occasionally step sideways - a genuinely different movement pattern, inspired by classic arcade shooters.
- A title screen and/or game-over screen: a proper opening screen (press a key to start) and/or a dedicated game-over screen with a final score, instead of just text overlaid on the gameplay - reuses your existing game_active flag and UI tools from Lessons 9-10. GameMaker's Room system and Draw GUI event (already used for your HUD) are a natural fit for a title/game-over screen.

If Something's Wrong

- Every wave still spawns exactly 5: confirm spawn_enemies()'s repeat loop actually uses (4 + obj_gamemanager.wave) and not a leftover literal 5.
- "You Win!" fires early or never: confirm enemies_remaining is reset using the SAME formula everywhere it's set - start_game(), obj_bullet's win-check else branch, and spawn_enemies()'s own loop count must all agree.
- Speed doesn't change between waves: confirm each new instance's enemy_speed is being set INSIDE spawn_enemies(), and that obj_enemy's Step event actually reads enemy_speed instead of a leftover literal 1.

Want More?

Student Resource List > GameMaker section, or manual.gamemaker.io